

Wisej.Web.Ext.QuillJS

A powerful rich text editor extension for Wisej.NET, built on top of QuillJS. This extension provides a seamless integration of QuillJS with Wisej.NET, offering rich text editing capabilities with support for custom formats, delta operations, and advanced text manipulation.

Features

- Rich text editing with a modern interface
- Delta-based operations for precise text manipulation
- Custom format support
- HTML import/export
- Link handling
- Selection management
- Cross-browser compatibility

Getting Started

Basic Usage

Add the QuillJSEditor to your form or page:

```
var editor = new QuillJSEditor();
editor.Dock = DockStyle.Fill;
this.Controls.Add(editor);
```

Text Formatting

Apply basic formatting to selected text:

```
// Apply bold formatting
editor.FormatText("bold", true);

// Apply italic formatting
editor.FormatText("italic", true);

// Apply underline formatting
editor.FormatText("underline", true);

// Remove all formatting
editor.RemoveFormat();
```

Working with Delta Operations

Delta operations provide precise control over text modifications:

```
// Create a new delta
var delta = new QuillDelta();

// Insert text with formatting
delta.Insert("Regular text ");
delta.Insert("bold text", new Dictionary<string, object> { ["bold"] = true });
delta.Insert(" and ");
delta.Insert("colored text", new Dictionary<string, object> { ["color"] = "red" });

// Apply the delta
await editor.SetDeltaAsync(delta);

// Get current delta
var currentDelta = await editor.GetDeltaAsync();
string deltaJson = currentDelta.ToJson();
```

HTML Manipulation

Import and export HTML content:

```
// Set HTML content
editor.Html = "<p>This is <strong>HTML</strong> content with"+
"<span style='color: blue;'>blue</span> text.</p>";

// Get HTML content
string html = editor.Html;
```

Link Handling

Handle links and link clicks:

```
// Handle link clicks
editor.LinkClick += (s, e) => {
    MessageBox.Show($"Link clicked: {e.Url}");
};

// Insert a link
var delta = new QuillDelta();
delta.Insert("Click here", new Dictionary<string, object> { ["link"] = "https://www.example.com" });
editor.UpdateContent(delta.ToJson());
```

Custom Formatting

Apply custom formatting to selections:

```
// Get current selection
var selection = await editor.GetSelection();
if (selection != null)
{
    var delta = new QuillDelta();
    delta.Retain(selection.Index);
    delta.Retain(selection.Length, new Dictionary<string, object>
    {
        ["background"] = "yellow",
        ["color"] = "black",
        ["bold"] = true
    });

    editor.UpdateContent(delta.ToJson());
}
```

Advanced Formatting

The editor supports sophisticated formatting operations:

```
// Apply custom background highlighting
editor.Format("background", "#FFEB3B");

// Insert formatted code blocks
var delta = new QuillDelta();
delta.Insert(codeExample, new Dictionary<string, object>
{
    ["code-block"] = true,
    ["background"] = "#f5f5f5",
    ["color"] = "#333333"
});
editor.UpdateContent(delta);

// Create custom formatted blocks
var delta = new QuillDelta();
delta.Insert("Custom Formatted Block", new Dictionary<string, object>
{
    ["blockquote"] = true,
    ["align"] = "center",
    ["color"] = "#2196F3",
    ["bold"] = true
});
editor.UpdateContent(delta);

// Apply text alignment
editor.Format("align", "center");
```

Extended Toolbar Configuration

The editor supports a rich set of toolbar options that can be customized:

```
editor.Toolbar = new object[] {
    new string[] { "bold", "italic", "underline", "strike" },
    new string[] { "blockquote", "code-block" },
    new string[] { "link", "image", "video", "formula" },
    new object[] { new { header = 1 }, new { header = 2 } },
    new object[] { new { list = "ordered" }, new { list = "bullet" }, new { list = "check" } },
    new object[] { new { script = "sub" }, new { script = "super" } },
    new object[] { new { indent = "-1" }, new { indent = "+1" } },
    new object[] { new { direction = "rtl" } },
    new object[] { new { size = new object[] { "small", false, "large", "huge" } } },
    new object[] { new { header = new[] { 1, 2, 3, 4, 5, 6, 0 } } },
    new object[] { new { color = new object[0] }, new { background = new object[0] } },
    new object[] { new { font = new object[0] } },
    new object[] { new { align = new object[0] } },
    new string[] { "clean" }
};
```

Event Handling

Monitor changes and interactions:

```
// Text changed event
editor.TextChanged += (s, e) => {
    Console.WriteLine($"New text: {e.Text}");
};

// Selection changed event
editor.SelectionChanged += (s, e) => {
    Console.WriteLine($"Selection: Index={e.Index}, Length={e.Length}");
};

// Delta changed event
editor.DeltaChanged += (s, e) => {
    Console.WriteLine("Content delta changed");
};
```

Advanced Usage

Custom Delta Operations

Example of complex delta operations:

```
var delta = new QuillDelta();

// Skip first 5 characters
delta.Retain(5);

// Format next 3 characters
delta.Retain(3, new Dictionary<string, object>
{
    ["bold"] = true,
    ["color"] = "red",
    ["background"] = "#f0f0f0",
    ["font"] = "monospace"
});

editor.UpdateContent(delta.ToJson());
```

Browser Compatibility

The extension is tested and works across modern browsers:

- Chrome/Chromium
- Firefox
- Safari
- Edge

Note: Special handling is implemented for Chrome-based browsers to ensure proper focus management and content editing behavior.

Best Practices

1. Always use async methods when working with deltas:

```
await editor.SetDeltaAsync(delta);
var currentDelta = await editor.GetDeltaAsync();
```

2. Handle selection before applying formats:

```
var selection = await editor.GetSelection();
if (selection != null && selection.Length > 0)
{
    // Apply formatting
}
```

3. Use the DeltaChanged event sparingly to avoid performance issues:

```
editor.DeltaChanged += (s, e) => {  
    // Handle with care - this event can fire frequently  
};
```

4. Use the TextChanged event to monitor text changes:

```
editor.TextChanged += (s, e) => {  
    // Handle text changes  
};
```

5. Use the SelectionChanged event to monitor selection changes:

```
editor.SelectionChanged += (s, e) => {  
    // Handle selection changes  
};
```

6. Use the LinkClick event to handle link clicks:

```
editor.LinkClick += (s, e) => {  
    // Handle link clicks  
};
```

7. Use the HtmlChanged event to monitor HTML changes:

```
editor.HtmlChanged += (s, e) => {  
    // Handle HTML changes  
};
```

8. Use the FocusChanged event to monitor focus changes:

```
editor.FocusChanged += (s, e) => {  
    // Handle focus changes  
};
```